Lab Guide 1

Introduction to Parallelism

After this lab session students should be able to:

- Collect basic performance metrics with `perf` and understand the correlation between low level performance metrics and the execution time [using the processor performance equation]:
  - Execution time = Instructions x Cycles Per Instruction x Clock Cycle Time (Texe = #I x CPI x Tcc)
- Identify the two fundamental types of parallelism: data-parallel and function-parallel
- Understand the impact of serial work parallelism limitation due to task dependencies.

Introduction

The first part of this lab session uses the `perf` tool to profile the execution of the Matrix Multiplication (MM) of type `double` case study (both were introduced in *Lab Guide 0*[1]). The main goal is to collect performance metrics and relate these metrics with the execution time.

The students are encouraged to use their own MM code, developed in lab session 0, but a simple MM is provided in annex 2).

Exercise 1 – Relation of low level metrics and the MM execution time

a) The default `perf stat` command collects a set of low level performance metrics. Run the matrix multiplication code with `perf stat ./a.out`. In the given output identify the main components of the processor performance equation.

b) What is the complexity of the MM (i.e., in big O notation, where N is the problem size and NxN is the matrix size)? What increase in execution time is expected when the N doubles? Which component of the performance equation is affected (e.g., #I, CPI or Tcc)? Use the `perf stat` to compare these metrics on a 512x512 and on a 1024x1024 matrix. (note: use -r 5 to get the average of five executions).

c) Measure the gain with the `O3` compiler optimisation level on a 512x512 matrix (compared with no optimisation, i.e., `O0`?) What component(s) of the performance equation is responsible for this gain?

Exercise 2 – Basic time profiling with perf

a) What is the complexity of each phase (e.g., function) in the MM case study (note: depending of your code, there should be several functions (e.g., phases): 1) matrix allocation; 2) matrix initialization; 3) matrix multiplication; 4) result validation). Which phase should be the starting point for optimisation? What is the restriction on the parallel execution of these phases?

b) Use the following commands to get a time profile with `perf` & view the profile:

```
# get a time-base profile, 1000 samples/s, write the result to perf.data file
srun --partition=cpar perf record -F 1000 -o perf.data ./a.out
perf report -i perf.data -n # view the report on the host
```

c) (*) Investigate the assembly of the MM function with the perf report for a fine-tuned hotspot identification.

d) (*) Collect and view a "call-**G**raph" profile by adding **`-g`** on `perf record`

[1] The resolution of this guide assumes the resolution of the previous guide.

Exercise 3 – Data parallelism and speedup analysis on a multithreaded execution

a) Develop a basic matrix multiplication using the class `std::thread` (see an example in annex 2). In this exercise we will take a **data-parallel approach**: each thread will compute a set of lines of matrix C.

b) Measure the speedup of using 4, 8 and 16 threads, when compared with the base sequential execution (on a 512x512 matrix). Note: for more consistent results, reserve all available PUs for execution with:

```
srun --partition=cpar --cpus-per-task=40 ./a.out
```

c) Investigate possible causes of the lack of ideal speedup in this case:

   i) By comparing low level metrics with `perf stat`

   ii) (*) By using `perf record` & `perf report` to get a more detailed profile.

Exercise 4 – Function parallelism, dependency graph and intrinsic speedup limitations

The Pearson correlation coefficient is a widely used technique of compute the degree of correlation between two series of values. In this exercise we will the following non-optimised implementation to illustrate function parallelism and dependencies (see complete code on annex 4).

```
double sum(double *v, int n) {
    double sum = 0.0;
    for (int i = 0; i < n; i++) sum += v[i];
    return(sum);
}
void deviation(double *vout, double *vin,  double mean, int n) {
    for (int i = 0; i < n; i++)  vout[i] = vin[i] - mean;
}
void mdeviation(double *vout, double *vin1, double *vin2, int n) {
    for (int i = 0; i < n; i++)  vout[i] = vin1[i] * vni2[i];
}
int main() {
    …
    double avg_x = sum(x,n)/n;
    double avg_y = sum(y,n)/n;
    deviation(diff_x, x, avg_x, n);
    deviation(diff_y, y, avg_y, n);
    mdeviation(product_diff, diff_x, diff_y, n);
    mdeviation(square_diff_x, diff_x, diff_x, n);
    mdeviation(square_diff_y, diff_y, diff_y, n);
    double numerator = sum(product_diff, n);
    double sum_square_diff_x = sum(square_diff_x, n);
    double sum_square_diff_y = sum(square_diff_y, n);
    double denominator = sqrt(sum_square_diff_x * sum_square_diff_y);
    double rxy = numerator / denominator;
    printf("Pearson correlation coefficient (r): %.4f\n", rxy);
}
```

$$r_{xy} = \frac{\sum_{i=1}^{n}(x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^{n}(x_i - \bar{x})^2}\sqrt{\sum_{i=1}^{n}(y_i - \bar{y})^2}}$$

a) Identify function calls that can (safely?) be executed in parallel. Draw an arrow between those functions calls.

b) Assume that all function calls take the same time to execute. What is the maximum speedup that can be obtained with this kind of parallelisation?

c) Look at the implementation of `sum`, `deviation` and `mdeviation`. What dependencies exist inside each function? How can we take advantage of that?

Annex 1: (simple) Instructions for using the search cluster:

a) **Copy local file to remote machine (don't forget the two points and point at the end) :**

```
scp <local file name> <student_id>@s7edu.di.uminho.pt:.
```

b) **Login:** `ssh <student_id>@s7edu.di.uminho.pt`

c) **Load the gcc environment:** `module load gcc/11.2.0`

d) **Compile:** `gcc -g -fno-omit-frame-pointer -O2 ...`

e) **Run:** `srun --partition=cpar perf stat -r 5 ./a.out  # assuming ./a.out on local directory`

Annex 2: Simple Matrix Multiplication code:

```c
#include<stdio.h>
#include<stdlib.h>

#ifndef N
#define N 512
#endif

double *A, *B, *C;

void alloc() {
   A = (double *) malloc(N*N*sizeof(double));
   B = (double *) malloc(N*N*sizeof(double));
   C = (double *) malloc(N*N*sizeof(double));
}

void init() {
  for(int i=0; i<N; i++) {
    for(int j=0; j<N; j++) {
      A[i*N+j] = rand();
      B[i*N+j] = rand();
      C[i*N+j] = 0;
    }
  }
}

void mmult() {
  for(int i=0; i<N; i++) {
    for(int j=0; j<N; j++) {
      for(int k=0; k<N; k++) {
        C[i*N+j] += A[i*N+k] * B[k*N+j];
      }
    }
  }
}

int main() {
  alloc();
  init();
  mmult();
  printf("%f\n", C[N/2+5]); # validade result
}
```

Annex 3: Example of std::thread usage:

Compile with: `g++ -lpthread ...`

```
#include<stdio.h>
#include<stdlib.h>
#include<thread>

#ifndef NT
#define NT 2
#endif

void printid(int id) {
   printf("id=%d\n", id);
}

int main() {
    std::thread tx[NT];

    for(int i=0; i<NT; i++)
        tx[i] = std::thread(printid,i);

    for(int i=0; i<NT; i++)
        tx[i].join();

    printf("End\n");

}
```

Annex 4: Pearson correlation coefficient complete code:

```
#include <stdio.h>
#include <math.h>

#define SIZE 100
double x[SIZE], y[SIZE];


// Calculate sum
double sum(double *v, int n) {
  double sum = 0.0;
  for (int i = 0; i < n; i++) {
    sum += v[i];
  }
  return(sum);
}
// Calculate deviation
void deviation(double *vo, double *vi,  double mean, int n) {
  for (int i = 0; i < n; i++) {
    vo[i] = vi[i] - mean;
  }
}
// Multiply deviations
void mdeviation(double *vo, double *vi1, double *vi2, int n) {
    for (int i = 0; i < n; i++) {
        vo[i] = vi1[i] * vi2[i];
    }
}

int main() {
    int n=SIZE;

    double diff_x[SIZE];
    double diff_y[SIZE];
    double product_diff[SIZE];
    double square_diff_x[SIZE];
    double square_diff_y[SIZE];

       ... // init X and Y

    double sum_x = sum(x,n);
    double sum_y = sum(y,n);
    deviation(diff_x, x, sum_x/n, n);
    deviation(diff_y, y, sum_y/n, n);
    mdeviation(product_diff, diff_x, diff_y, n);
    mdeviation(square_diff_x, diff_x, diff_x, n);
    mdeviation(square_diff_y, diff_y, diff_y, n);
    double numerator = sum(product_diff, n);
    double sum_square_diff_x = sum(square_diff_x, n);
    double sum_square_diff_y = sum(square_diff_y, n);
    double denominator = sqrt(sum_square_diff_x * sum_square_diff_y);
    double r = numerator / denominator;
    printf("Pearson correlation coefficient (r): %.4f\n", r);

    return 0;
}
```